CS-57 Human–AI Data Annotation, Provenance, and Dataset Evaluation Platform

Group Progress Report



Group Based 5703 Capstone Project

Group Members:

1. Michael Max (540559785)
2. Parth (550620451)
3. Jingwei Lin (520372766)
4. Hanchen Wang (520374346)
5. Yi Geng (520373453)
6. Dishank Aswal (540870299)
7. Kanishka Kathait (540870325)
8. Tim Chung (550512950)

Table of Contents

1.	**Progress & Achievements**	**1**
1.1	The project in brief	1
1.2	Summary: are we on track?	2
1.3	Who the platform is for	3
1.4	From user needs to sprints	4
1.5	Sprint by sprint: what was built and what was resolved	4
1.6	Front end: what each user can now do	5
1.7	Back end and data	7
1.8	Deliverables and the MVP	8
2.	**Obstacles**	**9**
2.1	Obstacles met so far	9
2.2	Challenges by sprint and story	10
2.3	Risks ahead	10
2.4	Dependencies	11
3.	**Deviation to Timeline**	**11**
3.1	The baseline	11
3.2	What moved, and why	11
3.3	Impact on the project	12
4.	**Milestones & Reporting**	**13**
4.1	Updated milestones	13
4.2	Remaining work, dependencies and the critical path	13
4.3	Who carries what	13
4.4	Slack, fallbacks and contingency	14
4.5	Reporting and communication	14
5.	**Acknowledgement of AI Use**	**14**
References		**15**
Appendix A. Figures and Tables		**16**
Appendix B. Identifiers and Terms		**28**
Appendix C. User Stories		**28**
Appendix D. Items Deprioritised or Tracked Outside Jira		**29**

1. Progress & Achievements

This report records team CS-57's progress at the start of Week 9 (8 October 2026): the work done **since our Week 5 proposal**, in four sprints from 7 September to 7 October, and the plan to the end of semester. Section 1 starts from the types of users of the platform and follows each of their needs through the tickets and sprints that met them, then describes the front end, the back end and the data, and shows that the minimum viable product (MVP) works. Sections 2 to 5 cover the obstacles met and risks ahead, the timeline deviations, the updated plan and our use of AI. Appendix A collects the detailed figures and tables, each linked from the text, and Appendix B explains the identifiers. Statements about the software refer to the main branch of USYD-CS-Capstone/hej at commit 1dcbcdf (7 October 2026).

1.1 The project in brief

Human Evaluation and Judgement (Hej) is Arc Intelligence's platform for producing labelled datasets for AI, and its core feature is data annotation. Annotators label items (for example, the topic of a news headline), reviewers accept or reject each label, and an expert settles any disagreement (Section 1.3). The team inherited a working but immature codebase: a Next.js web application and a FastAPI service with a relational database. The client was clear from kick-off that success means complete, defensible provenance rather than faster labelling: for every released item the platform must answer *how did this label get here* (who annotated it, what the AI suggested and whether it was accepted, who reviewed it, and how and under which guideline any disagreement was settled), and refuse to release data that cannot.

The client's brief groups the features into six pillars that organise our work: **Govern** (organisation management, roles and policies), **Assist** (AI suggestions), **Record** (provenance), **Adjudicate** (review and disputes handling), **Release** (gated, immutable releases) and **Evaluate** (repeatable measurement). Text annotation is the committed demonstration path; image, audio and video must keep working but are not extended.

How progress is measured. The team works in weekly Scrum sprints closed against a shared Definition of Done (Schwaber & Sutherland, 2020). Scope is a backlog of user stories (Cohn, 2004) grouped into eleven epics, A to K; for example, story D5 is *finalised work cannot be overwritten*. Each story is split into Jira tickets

(e.g., SCRUM-123), each delivered through a GitHub pull request (e.g., #12) approved by another member before merging. Work is estimated in units (u): one unit is two to three subtasks and equals one Jira story point. A story is done only when it is demonstrable in the running application, ships with tests, has any linked defect proven fixed by a test that fails on the old code (Feathers, 2004), has updated documentation and has been reviewed by someone other than its author.

1.2 Summary: are we on track?

The team remains on track to deliver every end-of-semester deliverable; one critical dependency is under close watch.

- **MVP complete, tested and working:** the core workflow agreed at Status Report 1 runs end to end on the client's data, from inviting members to exporting finalised items, and all automated and end-to-end tests pass on main (Section 1.8).
- **On schedule:** build work ends on 28 October (Week 11), followed by the technical report, decision records and demonstration; no proposed deliverable has been dropped.
- **Ahead of plan:** with the engineering foundations delivered on time (Section 3.2), the core workflow, AI assistance and governance started one to two weeks early.
- **About one week behind, and protected:** provenance and release, which extend the MVP to the full brief, form the critical path; the 1 October replan moved the provenance record from Week 11 to Week 9 so the release gate can build on it (Sections 3.3 and 4.2).
- **Recovered:** evaluation had not started at Status Report 2 (23 September), placing it behind schedule; the evaluation harness has since merged and the casebook is under way, returning this workstream to plan.
- **Unblocked:** the dispute record (E3/E4), paused 3–9 October on an open product question, now follows the team's two-level proposal; PR #48 is approved to merge in Week 9 (O4, R2).

In numbers (Table 1, Appendix A.1): 28 of 57 in-scope stories are complete (49%), 15 in progress and 14 not started; 49 pull requests have merged since 5

September, 22 of them in the last two weeks, each reviewed by another member; 1,283 backend and 400 frontend tests pass on main with none failing, up from 452 and 205 at Status Report 2; of the 30 inherited defects, 20 are fixed, 6 more are fixed with a closure check pending, and 4 are open; and 22 client questions have been answered, with 2 open.

Six stories were set aside by team decision on 14 September and are tracked outside Jira, not dropped (Appendix D): A1 (enforced branch protection) and A5 (one shared status vocabulary), as lower value than the remaining features, and K1–K4 (technical report, decision records, findings and demonstration), which are deliverables rather than platform features and are scheduled for 29 October to 1 November.

1.3 Who the platform is for

Hej serves seven types of user inside a client organisation (Table 2). Each holds one or more roles, and the server checks every action against those roles (Sandhu et al., 1996). The client, Arc Intelligence, owns the platform and decides product questions; it is a stakeholder rather than a user. Every story in the backlog is written for at least one of these users, which keeps the backlog tied to real needs (Cohn, 2004), and the rest of Section 1 follows those needs.

Table 2. Target users, what they need, and the epics that serve them

User (role)	Who they are	What they need from Hej	Epics
Administrator	Responsible for managing an organisation's members, including invitations, role assignments, and access permissions.	• Invite people with the right roles; • Change or remove roles; keep each project's work inside it. An invitee will be able to see pending invitations and accept one, gaining exactly the roles chosen	- G Roles - J Organisations
Task owner	Project managers who are responsible for configuring and managing projects, policies, and tasks	Configure review rules; load a dataset; choose the annotation surface; run the AI first pass; start, pause and complete tasks; reopen a finalised item; trust every figure; release data	- B Setting up work - C AI-Assisted First Pass - H Release
Annotator	Responsible for first-pass annotation (labelling)	Find available work; work without seeing colleagues' answers; never lose or have overwritten their own work	- B Setting up work - D Review
Reviewer	Accepts, corrects or rejects work	Judge a named person's submission; give reasons; escalate when unsure; never review their own work	- C AI-Assisted First Pass - D Review
Dispute Participant	Raises annotation disagreements for an expert to remediate	Raise disputes (Escalate) with reasoning attached for resolution by an expert (Note: A dispute participant should never dispute their own work)	- D Review - E Disputes
Expert (arbitrator)	Domain expert with the knowledge to remediate disputes	• View escalated tasks and items • Decide how data should be annotated • Record decision See escalated items; rule independently of the people involved; have the ruling recorded as its own decision	- E Disputes
Quality Lead	A senior staff member who assesses the released data and the evaluation harness results.	Repeatable runs, reviewer agreement, the AI's influence on people, and complete provenance. The quality lead may not hold an account on the platform, but relies on its outputs and is a key role in a client organisation.	- F Provenance - I Evaluation

1.4 From user needs to sprints

Table 3 traces each user's needs (columns 1 and 2) to user stories (column 3) and to the back-end and front-end work that met them (columns 4 and 5). Figure 1 shows the same work as a sprint calendar, one swim lane per user type.

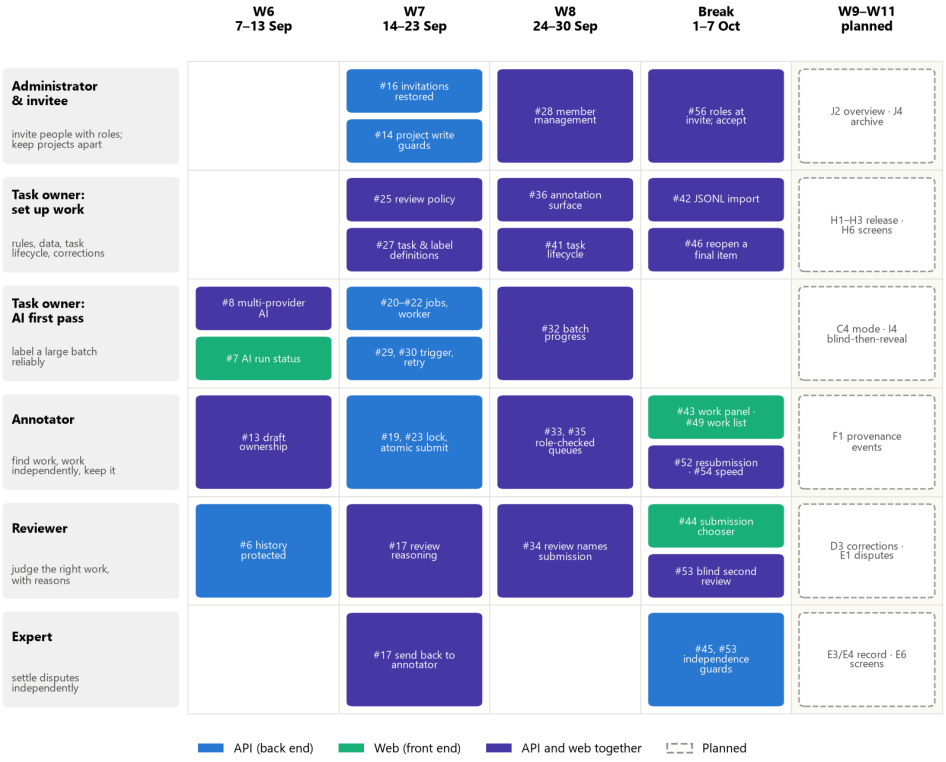


Figure 1. Merged pull requests by user and sprint (placed by merge date), with the work planned for Weeks 9–11

For example, an administrator needs to invite people into an organisation (G5, J3). The team first fixed the inherited invitation API, where every acceptance failed because the endpoint read the wrong token field and invitations were stored as already accepted (SCRUM-8, #16, W7). The member-management screens followed in the next sprint on the now-complete API (SCRUM-90, #28, W8). During the break, the team implemented the client's requirement that a role is assigned when an invitation is created and that the invitee accepts it on screen (SCRUM-115, #56). This is the general pattern: back-end capability (API and data model) first, then the screens, so most user needs span two sprints.

1.5 Sprint by sprint: what was built and what was resolved

Table 4 summarises each Jira sprint: its goal, planned and completed load, the features delivered and the problems solved. Many of those problems were defects

from our audit of the inherited code (30 catalogued); others surfaced during development or review. The challenges met inside each sprint are detailed in Section 2.2.

In brief, W6 (7–13 September) fixed governance and data integrity, closing the critical defects that let anyone edit another person's draft and let the legacy API rewrite approval history. W7 (14–23 September; 19 of 20.5 units) locked finalised work, made submission atomic, moved onto PostgreSQL with tests on both databases and built the AI batch worker. W8 (24–30 September; 20.5 of 22 units) routed work through role-checked queues and made each review name the submission it judges, so reviewers no longer see the wrong annotator's work. The mid-semester break (15 of 19.5 units, the rest in review) added the evaluation harness, answer versions, reopening and a blind second review, and cut a 200-item page from 404 requests to 6–8.

1.6 Front end: what each user can now do

This section describes, user by user, what the web application now lets each person do and what the team changed to support it. The screenshots (Figure 5 here; Figures 2–4 and 6–8 in Appendix A.1) were taken on 8 October from the main branch running locally on the client's AG News text dataset: one project with two tasks, one annotated by two people per item and one given an AI first pass by a local Llama 3 model. Names such as “Dana Lee”, “Frank Ng” and “Bob Johnson” are seeded test accounts.

Administrator and invitee. The members page (Figure 2) lists every user in the organisation with their account status, membership status and roles. Administrators can now create users, change roles and account status, and remove members (SCRUM-90, #28). As a functional requirement from the client, every invitation carries at least one role, and the invitee views and accepts pending invitations on screen, receiving exactly those roles (SCRUM-115, #56). Two front-end faults found in review were fixed on the way: reissuing an invitation could undo an acceptance, and switching organisation could let a late response overwrite the new organisation's data.

Project manager and task owner. The policy page (Figure 3) shows the organisation's defaults and the project's own review policy, marking each value as

Enforced (it already changes what the workflow allows) or *Recorded* (stored and shown until the story that enforces it lands, such as the Expert Gate, G2, in Week 10) (#25, #37). In task setup the owner chooses the annotation surface, such as classification or text spans, and the AI model (#36); defines labels and their meanings (#27); and uploads JSONL, JSON, CSV or text files, choosing per file which fields become the item that annotators and the AI see (#42). The task header carries activate, pause, resume and complete controls (#41); the AI run shows batch progress and each item's job status (#7, #32); and a finalised item can be reopened by the project owner with a reason (#46).

Annotator. The annotator's work list (Figure 4) shows only the items their role allows, with how many of the required human submissions each has: an item at "2 of 2 submitted" has its Annotate button greyed out, while one at "0 of 2" is open for work (#49). The work panel no longer shows colleagues' drafts or answers, keeps edits on returned work instead of reverting them, and reports any refusal in the server's own words (#43). The Annotate and Review pages now load in about half a second rather than three (#54).

Reviewer. The review panel (Figure 5) lists the submissions awaiting this reviewer on an item, by author; the reviewer picks one, and the decision is recorded against that submission only (#34, #44). The panel shows the task's review policy, here one approval with a justification for every decision, and offers accept, adjust, reject and escalate. On an AI-assisted task the AI's answer arrives for review in the same panel, labelled as AI-assisted (Figure 6). A sampled share of submissions goes to a second reviewer, who cannot see the first decision until their own is in (#53).

Expert. The dispute desk (Figure 7) lists items escalated to an expert with the reviewer's reason, and records the expert's decision, to finalise the item or send it back to the annotator, together with a note. The server refuses a decision from anyone who annotated, reviewed or routed the item, administrators included (#45, #53), enforcing separation of duty between roles (Sandhu et al., 1996). Once the expert rules, the item joins the task's finalised list (Figure 8) and can be exported. The client's refined three-outcome ruling and the dedicated dispute screens are Week 9–10 work.

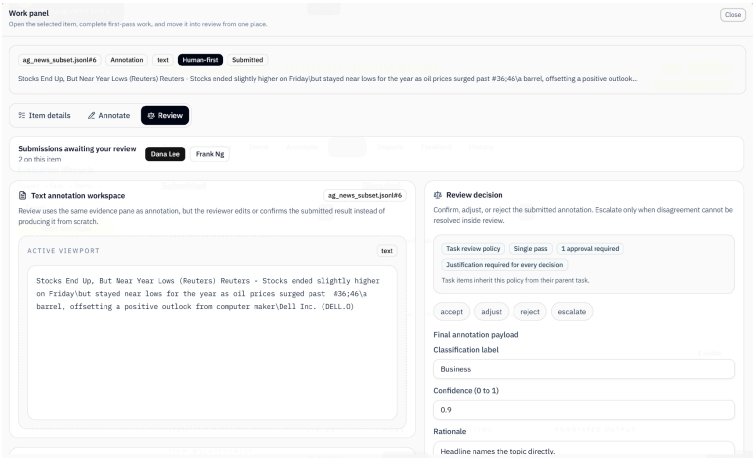


Figure 5. The review panel: two submissions await review on this item, and the reviewer picks one

1.7 Back end and data

The dataset. No new data is collected. The client supplied about 4 GB in four modalities (Table 5), each a task-aligned subset with a *source* folder of records or media files and a *handoff* folder of JSON files describing the task and its label list. Text arrives **structured**, as JSONL records that each hold an identifier, the text and gold labels; images, audio and video are **unstructured** media files. Raw client data is not copied into the database: items point to it, so the client's data stays where it lives.

AI: external models, not trained by us. The team does not train or fine-tune any model. The platform calls external large language models through one OpenAI-compatible client, reaching hosted APIs (OpenAI, Qwen via DashScope, DeepSeek, Doubao via Ark, and Groq) or a local model through Ollama, as in this report's screenshots, and records which provider and model produced each answer. The AI's output is one more author's answer that people must review, never ground truth, and a failed call is reported as a failure rather than a placeholder answer (C1–C3). Because LLM suggestions measurably shape annotators' judgements (Schroeder et al., 2025), Week 10 adds a blind-then-reveal protocol (I4) to measure that influence.

How the back end supports the front end. Every screen described in Section 1.6 rests on back-end work, usually merged a sprint earlier; Table 6 pairs each back-end change with the front-end feature it enables and the users it serves. For example, the role-checked queue API (#33, #35) is what lets an annotator's work list show “2 of 2 submitted” and grey out full items (Figure 4); making each review name the submission it judges (#34) is what the reviewer's submission chooser relies

on (Figure 5); and the AI job tables and background worker (#20–#22) feed the batch progress a task owner watches.

Back-end scripts and services used in operations. Some back-end pieces are not screens but are run by the team or an operator to make the platform usable; Table 7 lists them with what each enables for users. For example, init_data.py --reset rebuilds a demonstration organisation with an account for every role, so the client can sign in as an annotator, reviewer or expert and walk the whole workflow; the evaluation harness replays every casebook case through the real API, which is how the team checks behaviour before merging.

Records and quality. Every answer, whether from the AI, an annotator or a resubmission, is now kept as its own version, and the database enforces at most one authoritative version per item; reopening clears the marker but keeps every version (F3; #47). Each action also writes an audit entry with its actor and time: the entities, activities and agents a provenance record links (Moreau & Missier, 2013), on which the Week 9–10 release work builds. The evaluation harness (#39) runs scripted cases through the real API as real users holding real roles, and its four cases pass; draft PR #51 adds the casebook's adversarial categories and gold records. The platform runs on PostgreSQL 18 as well as SQLite, and CI runs the backend suite on both, with PostgreSQL in a non-UTC time zone to expose date-handling bugs (#24).

1.8 Deliverables and the MVP

Table 8 assesses each deliverable from the proposal against its own definition of done, with its planned and actual weeks, so each outcome can be read against the schedule in Section 3. Deliverables 1 and 7 are complete to date, 2 and 3 are largely built, and 4 and 5 are the focus of Weeks 9 to 11.

The MVP: complete, tested and working. At Status Report 1 (26 August) the client and the team agreed that success is first defined as a working, high-quality end-to-end annotation workflow for text, covering task setup, AI annotation, human review, role-based permissions and dispute resolution, with further functionality treated as extension work. That is our minimum viable product. Table 9 walks through it step by step, with the screen that shows each step working and the automated tests that check it; every test module named is on main, and every one passed when the suites were re-run for this report on 8 October.

Table 9. The MVP journey: where each step is shown working and how it is tested

Step	Who	Shown working	Tested by (backend modules; harness cases)
1 Invite members and choose their roles	Administrator	Figure 2	test_invitations, test_admin_iam_routes, test_role_separation_routes
2 Create a project and its review policy	Administrator, task owner	Figure 3	test_project_policy_enforcement, test_policy_resolver, test_governed_action_routes
3 Create a task, import a JSONL dataset, activate it	Task owner	Section 1.6	test_text_intake_service, test_text_intake_routes, test_task_lifecycle
4 AI first pass on every item	Platform (AI model)	Figure 6	test_ai_batch_worker, test_ai_preannotator, test_ai_run_progress; EV-002
5 Independent human annotation, up to the task's limit	Annotators	Figure 4	test_annotation_independence_privacy, test_task_required_annotators, test_draft_submission_atomicity
6 Review a named submission, with a justification	Reviewer	Figure 5	test_review_annotation_selection, test_review_justification, test_independent_second_review; EV-000, EV-003
7 Escalate; an independent expert rules	Reviewer, expert	Figure 7	test_adjudication_independence, test_review_actions; EV-001
8 Finalised item is locked and exported	Task owner	Figure 8	test_finalised_item_writes, test_task_exports, test_export_provenance

Verdict. The MVP is complete: all eight steps run through the user interface on the client's data, and the backend (1,283 tests), frontend (400 tests) and end-to-end harness (4 cases) all pass. Two refinements requested after the MVP was agreed are planned, not missing: the three-outcome expert ruling (accept, return or reject) is in PR #48, approved to merge in Week 9 (today the expert finalises or returns an item), and the gated release with a manifest is Deliverable 4 (Weeks 9–11; today export delivers finalised items). Provenance, release and evaluation extend the MVP to the full brief and take the remaining weeks.

2. OBSTACLES

Sections 2.1 and 2.2 cover the obstacles already met, which explain most timeline changes in Section 3; Sections 2.3 and 2.4 cover the risks and dependencies ahead, which shape the plan in Section 4. Each obstacle (O1–O9) and risk (R1–R10) names the deliverable and timeline it touches.

2.1 Obstacles met so far

Table 10 records nine obstacles; five shaped the project most. The inherited code was unsafe for multi-user work (O1): 30 defects, six critical, such as finalised work being silently overwritten. Treating these as technical debt to repay where it blocks delivery (Kruchten et al., 2012), the team agreed with the client to fix each defect inside the story it affects rather than in a separate clean-up phase; 20 are fixed and the two critical defects still open are scheduled for Week 9. Requirements were clarified only as features were built (O3), so tickets were rescoped mid-work and four stories added; a question log with needed-by dates and two written Q&A rounds answered every question blocking P0 work by 24 September. The dispute design

waited on an open client question (O4) from 3 to 9 October, until PR #48 adopted a two-level design that holds under either answer. At Status Report 2, provenance and evaluation had not started although the release gate depended on them (O7), so the break became an evaluation sprint and the provenance record moved from Week 11 to Week 9. Finally, the board sometimes ran ahead of the code (O8), so status reported to the client now follows code verified on main.

2.2 Challenges by sprint and story

Table 11 goes one level down: the concrete problems met while building particular stories, the sprint each arose in, how it was solved and the obstacle or risk it belongs to. Most were caught in code review or scenario testing before reaching main, the review practice adopted under O2. For example, the work-queue API was rescoped on the day it started, when the client ruled out assigning work to people, and review found that every non-empty queue returned an error because the route tests only checked refusals (W7–W8). The move to PostgreSQL meant taking ten test files off a hard-coded SQLite database so that CI genuinely tested both (W7). In the break, scenario testing found a 200-item task page sending 404 requests; one batched read cut it to 6–8 requests and about 560 ms.

2.3 Risks ahead

Table 12 is the risk register for the remaining weeks. Following Boehm (1991), each risk is rated for likelihood (L) and impact (I) as high, medium or low and plotted in Figure 9; each names what it affects, has more than one mitigation so that the plan does not rest on a single remedy, and has an owner and a trigger, the observable event that starts its contingency, both reviewed at the weekly meeting. The dependencies behind R1 and R2 are drawn in Figure 12 (Section 4.2).

Five risks combine medium likelihood with high impact. A late provenance record (R1) would delay the release gate, so its owner carries no other work and the gate's event-free checks are built first. A late or different client answer on disputes (R2) is absorbed by PR #48's two-level design. With no sprint after Week 11 (R3), that week is planned at 7.5u to absorb carry-over. Evaluation volume (R4) is spread across each story's own cases, with agreement reported as a chance-corrected coefficient rather than raw agreement (Artstein & Poesio, 2008). The handover documents (R7) have reserved days from 29 October.

2.4 Dependencies

Table 13 lists what the remaining work depends on, when each is needed and what happens if it is late. External dependencies, chiefly the client's answers on GitHub issues #38 and #40 and AI-provider availability, each have a fallback that does not need the outside party. Internally, the provenance record (F1) and authoritative answer (H4) must land in Week 9 for the release gate (H3) to follow in Week 10.

3. Deviation to Timeline

3.1 The baseline

The baseline is the Gantt chart in our proposal (Figure 10), refined at Status Report 1 (26 August) into a weekly plan that the client accepted as indicative, with changes raised at the weekly meeting: Week 5, user stories and the review workflow; Week 6, schema and contract validation; Weeks 7–8, the core annotation backend; Weeks 9–10, human review, roles and disputes; Week 11, an end-to-end run on the provided datasets; Week 12, hardening, documentation and the demonstration; Week 13, the final presentation and report. Since 19 September sprints run Thursday to Wednesday, closing after the Wednesday client meeting, so sprint W9 runs 8–14 October while teaching Week 9 began on 5 October; Week 7 absorbed the change and ran ten days, and the mid-semester break gained its own sprint (1–7 October), which the proposal's chart did not include.

3.2 What moved, and why

Figure 11 overlays the proposal's plan (grey) on what has been delivered (solid) and what is re-planned (hatched). Table 14 explains each deviation and links it to the obstacle that caused it (Section 2.1) or the risk it creates (Section 2.3).

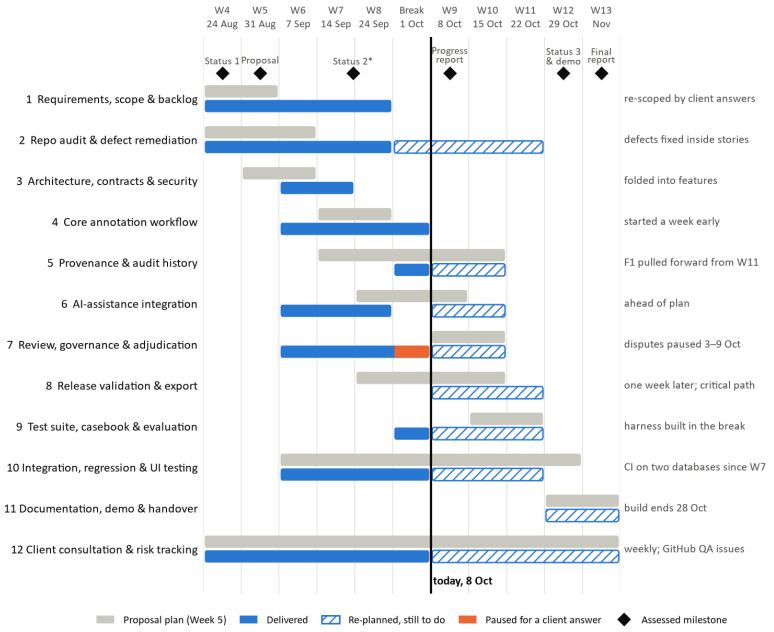


Figure 11. Planned versus actual timeline by workstream, as of 8 October

Unfinished work moves into the next sprint before new work is pulled in; everything carried into Week 8 and the break had merged by 7 October except SCRUM-109, now PR #60. Each sprint completed 77–93% of its planned load (Figure 13); the shortfall is work still in review at sprint close, not work abandoned.

3.3 Impact on the project

- **End date and scope unchanged.** Every proposal deliverable remains, build work ends on 28 October and assessed milestones keep their dates.
- **A critical path with no float after Week 9.** Read as a critical-path network (Kelley & Walker, 1959), the chain F1 → H3 → H2 → H6 → I5 → handover documents (Figure 12) decides the finish. Before 1 October, F1 sat in Week 11, after the release work that needs it, an order that could not have finished; after the replan the chain fits, but a slip in F1 delays the release gate unless the contingency in R1 is used. F2, H4 and H1 also feed the gate in Week 10, so they share its zero float (Table 16).
- **Slack lies in capacity, not in calendar time.** Because Week 11 is the last sprint, few tasks have spare weeks; instead Week 11 holds 7.5u against a capacity of 12–28u, and two members carry no counted work in it, which is where a slipped task lands (Section 4.4).

- **Two deliverable-level effects:** the dispute epic finishes in Week 11 rather than Week 10, and evaluation runs earlier than planned, which gives its measurements a week of slack.

4. Milestones & Reporting

4.1 Updated milestones

Table 15 (Appendix A.4) updates the proposal's milestone table week by week, following the template's convention: red text marks a change, struck-through text a task moved or replaced, and every task names its owner (past work by the member who delivered it, Week 9 from the Jira board, and Weeks 10–12 as proposed owners (*) to be confirmed at the weekly meeting). The main changes: the schema and contract audit was replaced by fixing contracts inside each feature; the canonical-annotation model became per-author answer versions (F3); the provenance record (F1) moved from Week 11 and the authoritative answer (H4) from Week 10, both to Week 9; the mid-semester break became a sprint for scenario testing and the evaluation harness; and Week 11 is now the last build sprint, with the end-to-end workflow run continuously on AG News data rather than as a one-off demonstration. Assessed milestones keep their dates; ticket-level detail is in Table 16.

4.2 Remaining work, dependencies and the critical path

Figure 12 (Appendix A.4) lays out the remaining stories by week with their dependencies and owners. It shows the critical path from Section 3.3, the further inputs from F2, H4 and H1 into the release gate (H3) and manifest (H2), and the dispute chain E1, E3/E4 → E6 → E5 alongside. Table 16 gives each ticket's owner, load, float and dependencies, where *float* is how many weeks a ticket can slip without delaying the end of build work on 28 October. Week 9 owners are on the Jira board; Week 10–11 owners marked with an asterisk are proposed from each member's earlier work, to be confirmed at the weekly meeting.

4.3 Who carries what

Table 17 turns the plan into each member's weekly load, which is how we check the timeline is achievable: everyone stays within the agreed 1.5–3.5u (Jingwei's 4u in Week 9 is by choice, with two tickets already in review), and the days after build work ends have named owners.

4.4 Slack, fallbacks and contingency

Figure 13 compares each sprint's planned load with what was completed, against the team's capacity band. Weeks 9 and 10 are planned near the middle of the band; Week 11 is deliberately light because no sprint follows it, and that spare capacity is the plan's fall-back space. Table 18 names, for each likely slip, the fallback and where the work goes. At every weekly meeting the team applies three replanning rules: (1) mark finished tickets done, verified against main; (2) move unfinished tickets into the next week *before* pulling in new work, and rebalance so each person carries 1.5–3.5u; and (3) if a client decision slips, the blocked group takes the fallback named in its roadmap row, pulled forward from a later week, and the blocked story returns at the first meeting after the answer.

4.5 Reporting and communication

The client meets the team every Wednesday evening to review progress since the last meeting, a demonstration of merged work and open questions; Week 9's meeting on 14 October reviews F1 and the answers to #38 and #40. Client questions go to the repository as GitHub issues labelled QA, where the issue thread is the record, and affected tickets and ADRs are updated the same day. Internally, the Jira board and contribution tracker are reconciled with main weekly, and the client-facing backlog page is refreshed after each reconciliation. The assessed milestones ahead are Status Report 3 with the demonstration (Week 12) and the final presentation, final report and technical report (Week 13).

5. Acknowledgement of AI Use

The team used Claude and ChatGPT as supporting tools. Each team member led their respective sections independently, using AI where helpful to clarify concepts and support implementation. All ideas, decisions and technical content are the team's own; each AI suggestion was reviewed, adapted or rejected by the responsible member. The team took full responsibility for learning and led all aspects of the assignment. All final deliverables reflect the team's own understanding and judgement.

References

Artstein, R., & Poesio, M. (2008). Inter-coder agreement for computational linguistics. *Computational Linguistics, 34*(4), 555–596. https://doi.org/10.1162/coli.07-034-R2

Boehm, B. W. (1991). Software risk management: Principles and practices. *IEEE Software, 8*(1), 32–41. https://doi.org/10.1109/52.62930

Cohn, M. (2004). *User stories applied: For agile software development*. Addison-Wesley.

Feathers, M. C. (2004). *Working effectively with legacy code*. Prentice Hall.

Kelley, J. E., & Walker, M. R. (1959). Critical-path planning and scheduling. In *Papers presented at the Eastern Joint IRE-AIEE-ACM Computer Conference* (pp. 160–173). ACM. https://doi.org/10.1145/1460299.1460318

Kruchten, P., Nord, R. L., & Ozkaya, I. (2012). Technical debt: From metaphor to theory and practice. *IEEE Software, 29*(6), 18–21. https://doi.org/10.1109/MS.2012.167

Moreau, L., & Missier, P. (Eds.). (2013). *PROV-DM: The PROV data model* (W3C Recommendation). World Wide Web Consortium. https://www.w3.org/TR/2013/REC-prov-dm-20130430/

Sandhu, R. S., Coyne, E. J., Feinstein, H. L., & Youman, C. E. (1996). Role-based access control models. *Computer, 29*(2), 38–47. https://doi.org/10.1109/2.485845

Schroeder, H., Roy, D., & Kabbara, J. (2025). Just put a human in the loop? Investigating LLM-assisted annotation for subjective tasks. In *Findings of the Association for Computational Linguistics: ACL 2025* (pp. 25771–25795). Association for Computational Linguistics.

Schwaber, K., & Sutherland, J. (2020). *The Scrum guide*. https://scrumguides.org/scrum-guide.html

Appendix A. Figures and Tables

The key figures and tables appear in the text; the detailed ones are collected here, grouped by the section that discusses them. Each reference in the text links to its figure or table, and each caption ends with a link back to that section.

A.1 Progress and achievements

Table 1. Progress in numbers, 7–8 October 2026 (back to Section 1.2)

Measure	Value	Context
User stories complete	28 of 57 in scope (49%); 15 in progress; 14 not started	63 stories in the backlog; 6 set aside by decision (see below)
Pull requests merged into main	49 since 5 September; 22 in the last two weeks (#29–#57)	Each reviewed by a member other than its author
Automated tests on main, re-run on 8 October	Backend: 1,283 passed, 9 skipped, 0 failed (plus 207 subtests). Frontend: 400 passed. Evaluation harness: 4 of 4 cases pass	452 backend and 205 frontend tests at Status Report 2 (23 Sep); CI runs the backend suite on SQLite and PostgreSQL on every push
Inherited defects (audit of 30)	20 fixed; 6 fixed with closure check pending; 4 open	Critical: 3 of 6 fixed, 1 pending check, 2 scheduled for Week 9
Client decisions	7 initial questions and 15 more in two written rounds answered; 2 open	Rounds of 21 and 24 September; open questions in GitHub issues #38 and #40
Sprint delivery	77–93% of planned load completed in each sprint	Shortfall is work still in review at sprint end (Sections 1.5, 3.2)

Table 3. User needs traced to stories, back-end and front-end tickets, sprints and pull requests (back to Section 1.4)

User (stakeholders)	Stakeholder Requirements	Story	Back-end: ticket (PR#, sprint#)	Front-end: ticket (PR#, sprint#)
Administrator, invitee	Bring people in with the right roles	G5, J3	SCRUM-8 invitation flow restored (#16, W7) SCRUM-90 member management (#28, W8);	SCRUM-115 roles at invitation, invitee accepts (#56, break)
Administrator	Keep each project's work inside it	G1, G3	SCRUM-44 roles drive permissions (#4); SCRUM-21 cross-project write guards (#14, W7)	None needed (enforced by the server)
Task owner	Set how work is reviewed	B2	SCRUM-20 review policy (#25, W7); SCRUM-50 policy resolved into task rules (#37, W8)	Policy page (#25, #37); policy shown in the review panel
Task owner	Load a dataset; choose how items are annotated	B5, B7, B8	SCRUM-34 all-or-nothing registration (#11, W7); SCRUM-111 JSONL import (#42, break); SCRUM-112 annotation surfaces (#36, W8)	Upload with per-file field selection (#42); surface and AI model chosen at setup (#36); label definitions (#27, W7)
Task owner	Start, pause and close a task	B4, B6	SCRUM-43 one completion rule (#26, W7); SCRUM-24 lifecycle (#41, W8)	Activate, pause, resume and complete controls (#41, W8)
Task owner	AI first pass on a large batch	C1–C3	SCRUM-6/7 multi-provider client (#8, W6); SCRUM-1/2/3/46 job tables, worker, retry, trigger (#20–#22, #29, #30, W7)	SCRUM-30 AI run status (#7, W6); SCRUM-5 batch progress (#32, W8)
Annotator	Find work; work independently; never lose it	D5, D6, D8	SCRUM-25 draft ownership (#13, W6); SCRUM-28/26 locked items, atomic submit (#19, #23, W7); SCRUM-48 role-checked queues (#33, #35, W8)	SCRUM-114 work panel (#43, break); SCRUM-93 available-work list (#49, break)
Reviewer	Judge the right person's work, with reasons	C5, D2, D4	SCRUM-29 history protected (#6, W6); SCRUM-31 structured reasons (#17, W7); SCRUM-27 review names its submission (#34, W8)	Review form with reasons (#17); SCRUM-113 submission chooser (#44, break)
Reviewer	An independent second opinion	D7	SCRUM-51 sampled blind second review, agreement report (#53, break)	Blind second review in the panel (#53)
Dispute participant, reviewer	Send a disagreement to an expert	E1, E5	SCRUM-51 disagreement recorded (#53, break); SCRUM-116 can't dispute own work (#45, break); SCRUM-101 dispute opened automatically, or held for a person to route (#48, W9); SCRUM-107 escalation rules (to do)	"Route to dispute" in the work panel; held cases on the dispute desk (#48, W9)

User (stakeholders)	Stakeholder Requirements	Story	Back-end: ticket (PR#, sprint#)	Front-end: ticket (PR#, sprint#)
Expert	Settle disputes independently	E2, D1, E3/E4	SCRUM-35 send back (#17, W7); SCRUM-116 and SCRUM-52 independence guards (#45, #53, break); SCRUM-99/100 adjudication record, Inconclusive outcome (#48, W9)	Dispute desk rebuilt on the new disputes (#48, W9); SCRUM-103 dispute screens (W10)
Project owner	Correct a finalised answer safely	D9	SCRUM-110 reopen with reason, old answer kept (#46, break)	Reopen action on a finalised item (#46)
Quality lead	Measure quality repeatably	I1–I5	SCRUM-68 harness (#39, break); I2–I5 in W9–W11	Measures reported from harness runs (W10)

Table 4. Jira sprints since the proposal: goal, load, what was built and problems solved (back to Section 1.5)

Jira sprint	Planned / done	Built	Problems solved
W6 Governance & data-integrity 7–13 Sep	not pointed	Draft ownership (D6); legacy review writes retired (D2); multi-provider AI client and AI run status (C1, C3); roles (G1)	Anyone could edit another's draft (critical defect 6); the legacy API could rewrite approval history (critical 8); failed AI calls produced fake annotations (20); environment and reset script broken (23, 24)
W7 Lock down review data 14–23 Sep	20.5 / 19	Invitations, project write guards, finalised-item lock and atomic submission, PostgreSQL with two-database CI, AI job tables, worker, retry and trigger, review policy, task and label definitions, one completion rule	Finalised work silently overwritten (critical 2); submission not atomic (11); cross-project writes (9); invitations broken (17, 18); invalid statuses saved and a 409 reported as 500 (10, 29); dataset registration not transactional (16); no send-back (15); a 1,000-item AI batch held one request open for hours
W8 Route work, record decision 24–30 Sep	22 / 20.5	Role-checked work queues with a per-task submission limit; review names its submission; policy governs review; annotation surfaces; task lifecycle; member management; AI batch progress	Reviewer shown the wrong annotator's work (3); a task could never leave draft (28); a task with items could not be deleted and returned 500 (30); every non-empty queue returned 500 (found in review); reissuing an invitation could undo an acceptance
Mid-semester Break 1–7 Oct	19.5 / 15	Evaluation harness; answer versions; owner reopen; blind second review and adjudicator independence; work panel, submission chooser and available-work list; JSONL import; invitee acceptance	Self-adjudication on disputes (critical 7, fix merged); a reopened item flipped back by the next read; resubmission allowed on approved work; review decisions the queue would not offer; Save draft reverting a returned item's edits; a 200-item page sending 404 requests (2,969 ms); duplicated history entries (22)
W9 Trace decisions (current) 8–14 Oct	20 / in progress	Provenance events (F1), versioned guidelines (F2), corrections (D3), one authoritative answer (H4), release artefact (H1), automatic disputes (E1), mode on every item (C4), repeatable runs (I1)	To solve: reviewer corrections not saved (critical 5); conflicting exported answers (critical 4)
W10, W11 15–28 Oct	20.5, 7.5	Release gate and manifest, dispute screens, Expert Gate, item timeline, measures; real-data screens and integrity checks	Planned (Tables 15–17)

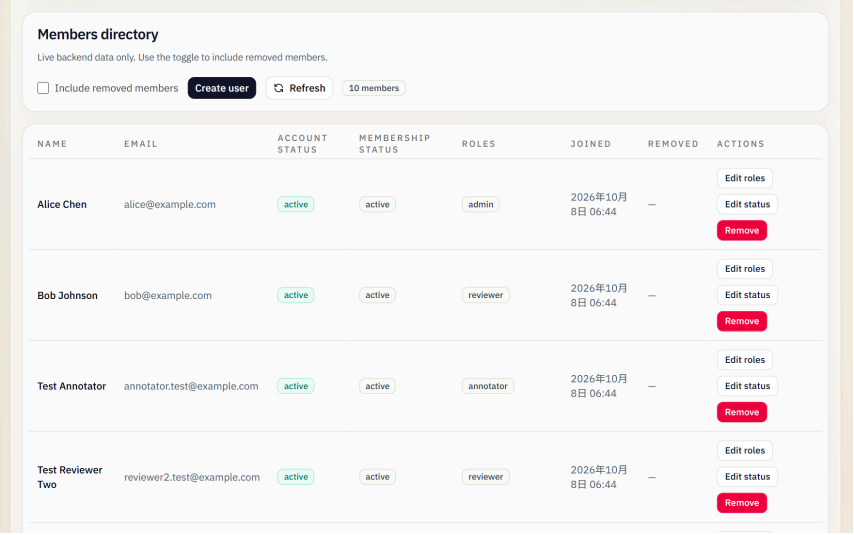


Figure 2. The administrator's members page: roles, status and the actions available on each member (back to Section 1.6)

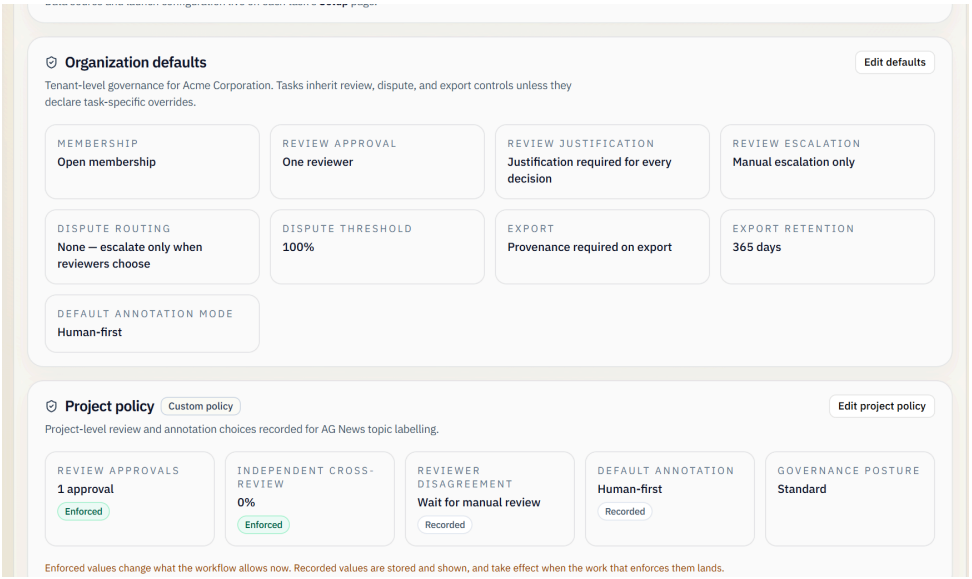


Figure 3. Organisation defaults and the project policy, distinguishing enforced from recorded rules (back to Section 1.6)

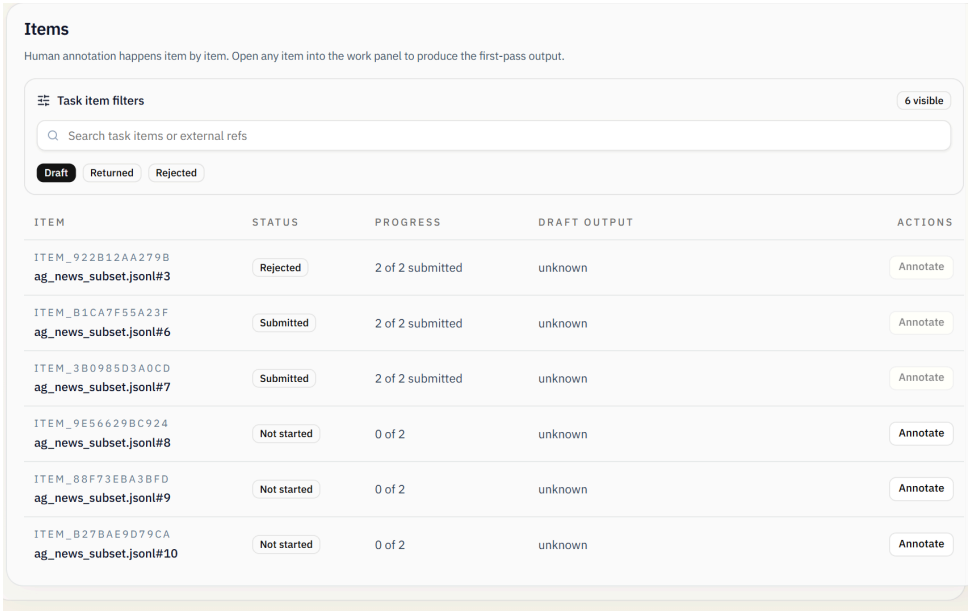


Figure 4. An annotator's work list: items that already have their two submissions cannot be taken (back to Section 1.6)

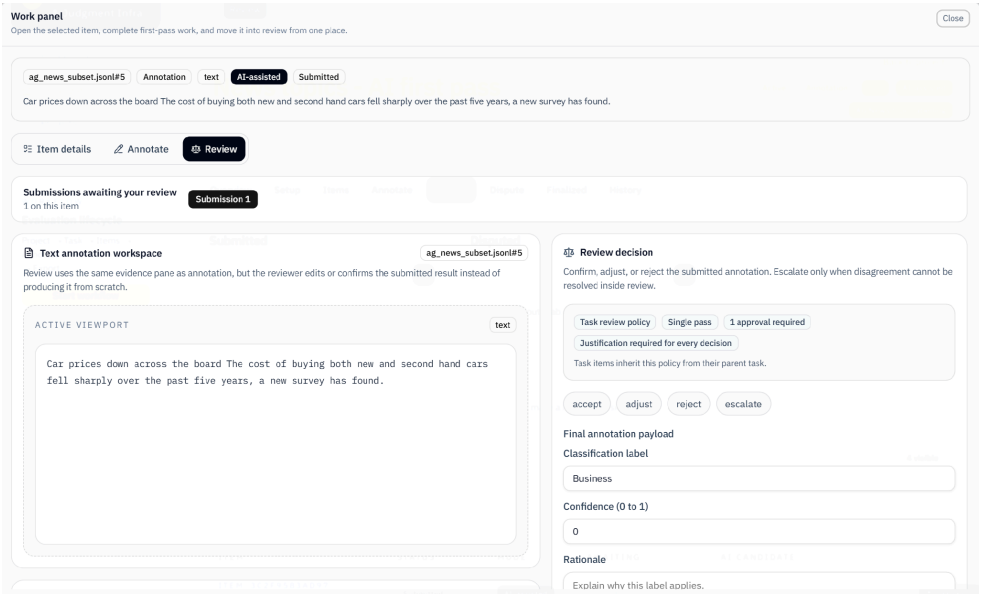


Figure 6. A reviewer judging the AI's first pass (Llama 3 labelled this headline "Business") (back to Section 1.6)

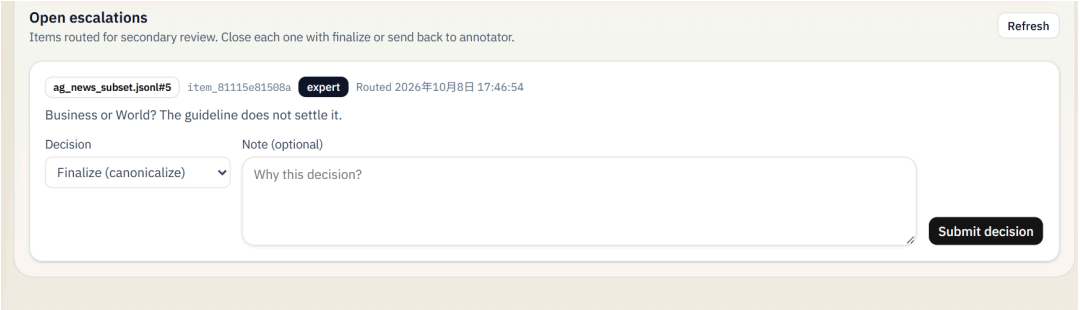


Figure 7. An escalated item waiting for an expert's ruling, with the reviewer's reason for escalating (back to Section 1.6)

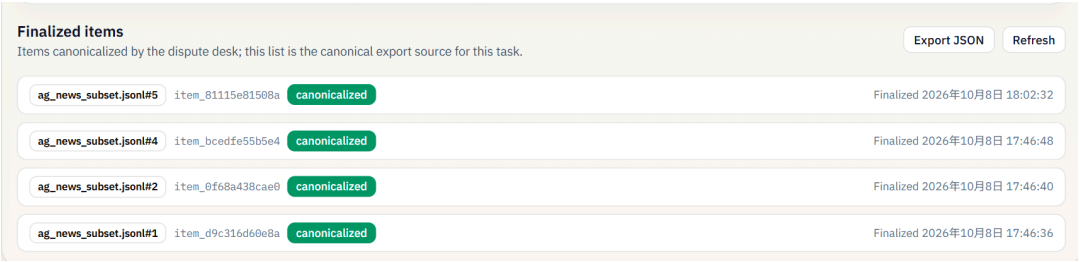


Figure 8. Finalised items, including item #5 after the expert's ruling, ready for export (back to Section 1.6)

Table 5. The client's datasets and how each kind enters the platform (back to Section 1.7)

Modality	Datasets (task, size)	Form	How it enters Hej
Text	AG News (topic classification), Few-NERD (named entities), SemEval-2010 Task 8 (relations), Civil Comments (moderation judgement); 200 items each	Structured: JSONL records with id, text and gold labels; JSON label lists	JSONL import (#42): one item per record; the task selects which fields annotators and the AI see, so gold labels stay hidden; the source file's hash is recorded on each item
Image	BoneBreak (classification), Retail Product Checkout (inventory judgement); 200 files each	Unstructured image files	Upload: one item per file, referenced by pointer
Audio	LibriSpeech (transcription), MInDS-14 (intent); 200 WAV files each	Unstructured audio	Upload; transcription through an external speech API (whisper-1)
Video	UCF101 (action recognition, 202 files), MOT17 and UOT32 (object tracking)	Unstructured video and tracking sequences	Upload; kept working but not extended, since text is the demonstration path

Table 6. Back-end changes and the front-end features they enable (back to Section 1.7)

Back-end change (API, data model)	Front-end feature it enables	Users
Invitation lifecycle and role assignment (#16, #56)	Members page: invite with roles, pending invitations, accept (#28, #56)	Administrator, invitee
Capabilities from current roles; cross-project write guards (#4, #14)	Pages offer only what a role may do; refusals show the server's reason	Everyone
Policy resolver: project policy into task rules (#25, #37)	Policy page with enforced and recorded values; policy in the review panel	Task owner, reviewer
Text intake service: one item per record, field selection, source hash (#42)	Upload with per-file field selection	Task owner
Annotation-surface catalogue (#36)	Setup chooses the surface; the editor follows it	Task owner, annotator
Task lifecycle endpoints and guards (#41)	Activate, pause, resume and complete controls	Task owner
AI job tables, background worker, retry and dead-letter (#20–#22, #29, #30)	Batch progress and per-item job status (#32); AI answer in the review panel	Task owner, reviewer
Role-checked queues, submission limit, independence rules (#33, #35)	Available-work list with "2 of 2"; work panel without colleagues' answers (#43, #49)	Annotator, reviewer
A review names the submission it judges (#34)	Submission chooser (#44)	Reviewer
Cross-review sampling and blind second review (#53)	Second reviewer sees the first decision only after deciding	Reviewer, quality lead
Escalation routing and decision guards (#45, #53)	Dispute desk refuses non-independent experts; nobody can send their own work to dispute	Expert, dispute participant
Reviewer disagreement detected; dispute opened automatically or held for routing, opener recorded (#48)	Held cases on the dispute desk; "Route to dispute" with a justification	Dispute participant, reviewer
Adjudication recorded as its own decision; Inconclusive outcome; reconciliation of several accepted answers (#48)	Dispute desk: Accept, Return or Reject with a reason; Inconclusive shown as final	Expert, task owner
Answer versions with one authoritative answer (#47)	Finalised list and export carry every author's answer	Task owner
Reopen endpoint keeping the old answer (#46)	Reopen action on a finalised item	Project owner
One batched read of a task's drafts, new indexes (#54)	Annotate and Review pages load in ~0.56 s instead of ~3 s	Annotator, reviewer

Table 7. Operational scripts and services, and what they enable (back to Section 1.7)

Script or service	What it does	What it enables, and for whom
init_data.py --reset	Rebuilds the database with two demonstration organisations, eight accounts covering every role, and sample projects, tasks and items	The team and the client can sign in as any role and walk the whole workflow; used for demonstrations and manual tests
AI batch worker (HEJ_AI_EXECUTION_MODE=worker)	A separate process that claims AI jobs, calls the model, retries transient failures with back-off and dead-letters permanent ones	Task owners start an AI run and watch its progress instead of waiting on one long request
Evaluation harness (python -m evaluation run)	Runs every casebook case through the real API as real users in a fresh database and writes a JSON report	Quality lead and developers check behaviour before merging; the basis of the Week 10–11 measures
Docker Compose with PostgreSQL 18	Starts the database, the API and, optionally, the AI worker	Every member and the client run the same production-like environment
Schema migration at start-up	Adds new columns to an existing database	Existing demonstration data survives upgrades
CI workflow	Runs the backend suite on SQLite and PostgreSQL on every push	Reviewers see whether a pull request breaks either database

Table 8. Status of the seven proposal deliverables on 8 October (back to Section 1.8)

Deliverable (planned → actual weeks)	Status	Evidence and what remains
1 Engineering foundations (W5–W6 → W5–W7)	Achieved	Platform builds and runs from a clean clone; test suites restored and made truthful (PR #2); hard-coded credential removed; every change merged through a reviewed PR; CI runs the backend suite on SQLite and PostgreSQL on every push (#24, W7). Gap: GitHub branch protection is not switched on, so review is a team rule rather than enforced.
2 Core end-to-end workflow for text (W7–W10 → W6–W10)	MVP achieved	Project → policy → task → JSONL import → AI first pass → human annotation → review → escalation → expert ruling → export runs through the user interface on the client's AG News data (Section 1.5). Remaining: automatic disputes (E1, W9), the client's refined adjudication record (E3/E4, PR #48 approved, merging in W9) and the dispute screens (E6, W10).
3 Provenance and governance (W9–W10 → W6–W10)	Governance achieved; provenance in progress	Roles, invitations, project boundaries, self-review guards and independent second review are merged. Every answer is now its own version with at most one authoritative (F3). Remaining: the provenance event record (F1) and versioned guidelines (F2) in W9; item timeline and actor kinds (F4, F5) in W10.
4 Release validation and export (W10–W11 → W9–W11)	Not started (on plan)	Export of finalised items works and follows one completion rule (B6). Week 9: one authoritative answer per item (H4) and the immutable release artefact (H1). Week 10: the release gate (H3) and manifest (H2). Week 11: release, policy and history screens on real data (H6).

Deliverable (planned → actual weeks)	Status	Evidence and what remains
5 Evaluation harness and casebook (W10–W11 → break–W11)	Ahead of plan	Harness merged (#39); its four end-to-end cases pass. Casebook categories and adversarial cases in a draft PR (#51). Repeatable runs in W9; 40 cases and agreement/AI-influence measures in W10; integrity checks and findings record in W11.
6 Documentation and handover (W12 → 29 Oct – 1 Nov)	In progress	Architecture decision records are written as each client decision lands; the technical report, per-member decision records and demonstration are scheduled for 29 October – 1 November (Table 17).
7 Assessed artefacts (W5, W9, W13 → unchanged)	On schedule	Proposal submitted (W5); Status Reports 1 and 2 held with the client on 26 August and 23 September; this progress report (W9); Status Report 3 with demonstration (W12); final presentation and reports (W13).

A.2 Obstacles

Table 10. Obstacles met, their effect, our response and where each stands (back to Section 2.1)

ID	Obstacle	Effect (deliverable; timeline)	What we did	Status
O1	Inherited code unsafe for multi-user work: 30 defects, 6 critical (overwritten finalised work, editing others' drafts, self-approval, conflicting exported answers); failing and vacuous tests; a live API key in source	Almost every feature touched defective code; a separate clean-up phase would have used Weeks 6–7 (Deliverables 1, 2; Table 14 row 2)	Agreed with the client (26 Aug) to attach each defect to the story whose behaviour it affects; fixed only the defects that blocked building and testing first; each fix ships a test that fails on the old code	20 fixed, 6 awaiting closure check, 4 open and scheduled
O2	No review discipline in the inherited repository; changes had been overwritten	Regression risk with eight people working in parallel (Deliverable 1)	Every change through a PR reviewed by another member; CI on both databases from Week 7	49 PRs merged with review; branch protection still not enforced (A1 set aside)
O3	Requirements clarified only as features were built per-author answer versions instead of one canonical answer, and no assignment of work to people (17 Sep); policy changes as new versions (21 Sep); expert adjudication made final (24 Sep)	Tickets rescoped mid-work (the queue API on the day it started); four stories added (B7, B8, D9, A6); D3, F3 and H4 blocked until 24 Sep (Deliverables 2–4; Table 14 rows 1, 5)	Question log with deadlines and blocked stories; two written Q&A rounds; every blocked roadmap group names a fallback; decisions recorded as ADRs and tickets updated the same day	Every question that blocked P0 work answered by 24 Sep
O4	Two product questions open: does a dispute concern the whole item or one answer, and may an expert merge answers (issue #40); who reworks an AI first pass a reviewer returns (#38)	E3/E4 (PR #48) paused 3–9 Oct; returned AI items are stranded on main (Deliverables 2, 4; Table 14 row 7)	Built the parts that do not depend on the answer (separate adjudication record, independence check, row lock); posted both options neutrally on #40; raised at the 7 Oct client meeting; on 9 Oct #48 went ahead on the team's two-level proposal posted on #40	#48 approved, merging in W9; #38 and #40 still open (R2)
O5	Infrastructure changes mid-sprint: moving to PostgreSQL meant ten test files hard coded to SQLite, no migration tool for schema changes, and a period when GitHub Actions would not run because of an account spending limit	One member's Week 7 went to infrastructure (3u); everyone updated local settings on merge day (Deliverable 1; Table 14 row 3)	One shared test fixture; CI on both databases; one agreed migration order per week and reset development databases; full suite run locally as evidence while CI was down	Resolved; procedure continues (R5)
O6	Parallel work on the same code: three tickets adding refusals to draft submission; four tickets changing the review path in the break	Merge conflicts, and refusals in the wrong order give users the wrong reason (Deliverable 2)	Merge order agreed on the first day of each week; stacked PRs (#43 → #44); paired authors review each other's PRs; refusal order documented with the API (#35)	Managed weekly (R10)
O7	Sequencing: at Status Report 2 (23 Sep) provenance and evaluation, both heavily weighted deliverables, had not started, and the release export (W10) depended on a provenance record scheduled for W11	Release gate could not be finished; evaluation would be late and thin (Deliverables 3–5; Table 14 rows 5, 8, 9)	Turned the mid-semester break into a sprint for the evaluation harness and scenario testing; moved the provenance record from W11 to W9 (replan of 1 Oct); pulled answer versions (F3) into the break	Harness merged; F1 under way in W9 (R1)
O8	Records drifting from the code: a ticket marked Done although its interface existed on no branch; D2 marked complete with two subtasks unbuilt	Progress over-reported to the client (Deliverable 7, status reporting)	Roadmap status follows code verified on main; a tracker row counts only after its review passes; differences raised at the weekly sync	Ongoing control (R6)
O9	Performance found late: a 200-item task page made 404 requests (2,969 ms), and every read repaired data with a write, which also caused a workflow bug	Dataset-sized tasks unusable for the demonstration (Deliverables 2, 7 demo; new story A6)	Batched read (#54) and bug fix (#50); "reads never write" scheduled for W10 (SCRUM-121)	6–8 requests, about 560 ms

Table 11. Technical challenges met while building, by sprint and story (back to Section 2.2)

Sprint	Story, ticket (owner)	Challenge	How it was solved	Link
W6	D6, SCRUM-25 (Hanchen)	AI-created drafts had no owner, so "only the author may edit" had to keep unclaimed AI drafts usable without opening a back door	Ownership guard with a required viewer; unclaimed AI drafts can be claimed, never edited by others (#13)	O1
W7	Infrastructure, SCRUM-94 (Parth)	Ten test files hard-coded SQLite, so a PostgreSQL CI job would have tested nothing; the SQLite settings a second writer needs (WAL, busy timeout) were missing; docker compose up hung	One shared test fixture taking the database from configuration; the settings added with tests; start-up fails fast and names what is missing (#24)	O5
W7	C2, SCRUM-2 (Michael)	A worker whose lease had expired could still write and overwrite another worker's result; a permanent configuration error took about 15 minutes to surface; compose ran the worker on SQLite while the API used PostgreSQL	Lease ownership checked on every write, retry limit enforced, permanent errors dead-lettered at once, worker moved onto the shared fixture (#22, #30)	O5
W7–W8	D8, SCRUM-48 (Kanishka; fixes Hanchen)	Rescoped the day it started, when the client ruled out assigning work to people; independence had to become part of the queue query; every non-empty queue returned 500 because route tests only checked refusals; CI would not start (spending limit)	Per-submission review queue with independence in the query; route tests with real roles; full suite run locally as evidence (#33, #35)	O3, O5
W8	J3, SCRUM-90 (Tim)	Reissuing an invitation could undo an acceptance; a late response after switching organisation overwrote the page	Renewal only while an invitation is pending; stale responses ignored (#28, second review round)	O6
W8	B4, SCRUM-24 (Dishank)	The web app could not activate a task, so nothing created in the interface could be worked on; queues still offered work on draft and paused tasks	Lifecycle controls added to the web app; queues respect task state; the AI first pass moved to activation (#41)	O3
W8	B7, SCRUM-111 (Yi)	Each item carried an unauthenticated link to the whole source file, gold labels included	Source kept private; items carry only the selected fields and the source hash; decision recorded in an ADR (#42)	O1
W8	C2, SCRUM-46 (Michael)	A dead-lettered item could never be re-run; the AI's annotation had no author	Re-run path added; provider and model recorded on the annotation; an AI author identity moved to F3 (#29, #47)	O3
Break	D7, SCRUM-51/52 (Parth)	Blindness never ended: a reviewer stayed blind to decisions after the round closed, and an adjudicator who could review was blind to the dispute; the agreement report leaked an open round's verdicts	Raised in review and addressed before the PR was approved by three reviewers and merged; one shared rule now decides both the adjudicator queue and the decision (#53)	O6
Break	D9, SCRUM-110/120 (Jingwei)	A reopen reason over 2,000 characters was a 500 on PostgreSQL; a reopened item was flipped back to expert send-back by the next page read	Length validated; status resolution respects the reopen (#46, #50); reads that write moved out in W10 (SCRUM-121)	O9
Break	A6, SCRUM-119 (Hanchen)	A 200-item Annotate page sent 404 requests and ran 6,882 SQL statements (2,969 ms); no single statement was slow	One batched read of items and drafts, with indexes: 6–8 requests, about 560 ms (#54)	O9
Break	E3/E4, SCRUM-99/100 (Yi)	Whether a dispute concerns one answer or the whole item was never settled with the client	Paused 3–9 Oct while the parts that do not depend on the answer were built; #48 then adopted the team's two-level proposal posted on #40 and was approved (9 Oct)	O4, R2

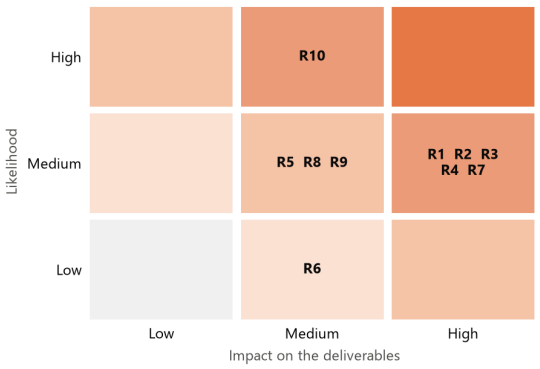


Figure 9. Risk heat map: the five medium-likelihood, high-impact risks drive the contingency plan (back to Section 2.3)

Table 12. Risk register for Weeks 9–13 (back to Section 2.3)

ID	Risk and what it affects	L	I	Mitigations	Owner; trigger
R1	The provenance event record (F1) slips. The release gate (H3), manifest (H2), item timeline (F5), real-data screens (H6) and integrity checks (I5) all read its events. Affects Deliverables 3, 4; W10–W11	M	H	(a) F1's owner carries no other counted work in W9. (b) The event format is fixed early in the week so the W10 release work can build against it. (c) If F1 is late, W10 builds the gate's checks that need no events first (unreviewed, superseded and disputed items) and adds the provenance check when F1 merges. (d) W11's spare capacity absorbs the remainder	Parth; F1 not merged when W9 closes (14 Oct)
R2	Client answers on #40 and #38 arrive late or change scope. E1 → E6 dispute screens (W10) → E5 (W11), and part of H4, depend on them. Affects Deliverables 2, 4; W10–W11	M	H	(a) PR #48 follows the team's two-level proposal, so the dispute work no longer waits for the answer. (b) Keep the two options' differences inside one service. (c) If the client's answer differs, the change becomes a follow-up ticket, recorded as a reversible ADR. (d) H4 first covers the agreed cases: one authoritative answer per item, none for an ambiguous item	Hanchen; the client's answer on #40 differs from the proposal
R3	No room after Week 11: build ends 28 Oct and no sprint follows. Affects Deliverables 2–5; W11	M	H	(a) W11 planned at 7.5u against a capacity of 12–28u. (b) P0 work first (H6). (c) J1 is the first story dropped, then P1 scope by agreement with the client. (d) Carry-over is planned before any new work	Whole team; W11 load above 12u after W10 carry-over
R4	Evaluation volume and quality: 40 cases and agreement and AI-influence measures by W10, findings by W11. Affects Deliverable 5; W10–W11	M	H	(a) Every story adds cases for its own behaviour. (b) The harness already drives the real API. (c) Scripted AI answers make runs repeatable. (d) Agreement reported with an established chance-corrected coefficient, never a single unexplained score	Hanchen, Tim, Parth, Michael; casebook below 40 cases when W10 closes (21 Oct)
R5	Several schema changes in W9 (F1, F2, C4) with no migration tool. Affects Deliverable 3; W9	M	M	(a) One migration order agreed on day one. (b) Development databases reset afterwards. (c) CI on both databases catches divergence	Parth, Michael, Jingwei; a migration conflict in CI
R6	Records overstate progress to the client. Affects Deliverable 7; every sprint	L	M	(a) Status from code verified on main. (b) Tracker review gate. (c) Weekly reconciliation of board, tracker and code	Hanchen; board or tracker differs from main at the weekly sync
R7	Handover deliverables (technical report, per-member decision records, findings, demonstration) are not on the Jira board. Affects Deliverables 6, 7; 29 Oct – W13	M	H	(a) 29 Oct – 1 Nov reserved for them, with owners (Table 17). (b) ADRs written with each decision feed the decision records. (c) I5's findings record feeds the findings. (d) The demonstration is rehearsed on the real screens once H6 lands, on this report's AG News set-up	Hanchen; no report outline by 28 Oct
R8	The client requires project-level isolation for reads as well as writes; writes are guarded, reads are not yet. Affects Deliverable 3; W11	M	M	(a) Raised as a product decision. (b) Built in W11's spare capacity if possible. (c) Otherwise recorded as a known limitation in the technical report	Hanchen; client confirms it is needed for the demo
R9	AI providers are non-deterministic or unavailable during evaluation or the demonstration. Affects Deliverables 5, 7	M	M	(a) The harness uses scripted AI answers. (b) The demonstration can run on a local Ollama model, as in this report. (c) Failures are shown, never disguised	Michael; a harness or demo run fails on a provider error
R10	Integration conflicts on the review path: three open PRs (#48, #58, #60) change the same functions. Affects Deliverable 2; W9	H	M	(a) Agreed merge order. (b) Whichever merges second rebases. (c) Authors review each other's PRs	Jingwei, Parth, Yi; a PR needs a rebase after another merges

Table 13. External and internal dependencies (back to Section 2.4)

Dependency	Type	Needed by	Status (8 Oct)	If it is late
Client answer on dispute scope (issue #40)	External	E3/E4 now; E6 in W10	Open; #48 no longer waits (9 Oct)	R2: a different answer becomes a follow-up ticket
Client answer on reworking a returned AI first pass (#38)	External	AI-assisted tasks; the demo	Open since 3 Oct	Demonstrate rework on human-first tasks; record the gap as a known limitation
Weekly client meeting	External	Each sprint review	Held every Wednesday	Questions continue asynchronously on GitHub
Client datasets with gold labels	External	I2–I4 measures (W10)	Available locally	Not needed
AI providers (hosted keys, local Ollama)	External	AI runs; I4 (W10); the demo	Local Ollama verified on 8 Oct	R9: scripted AI answers
GitHub Actions CI	External	Every PR	Running	Full suite run locally and attached to the PR
F1 provenance events	Internal	H3, H2, F5 (W10); I5 (W11)	In progress (Parth)	R1 contingency
H4 authoritative answer; H1 artefact	Internal	H3, H2 (W10); H6 (W11)	Not started (W9)	Zero float: owners report mid-week at the client meeting
D7 disagreement comparison	Internal	E1 (W9)	Done (#53)	Not needed
I1 repeatable runs	Internal	I3, I4 (W10)	In progress (Hanchen)	I3/I4 start on single runs, add comparison later

A.3 Deviation to timeline

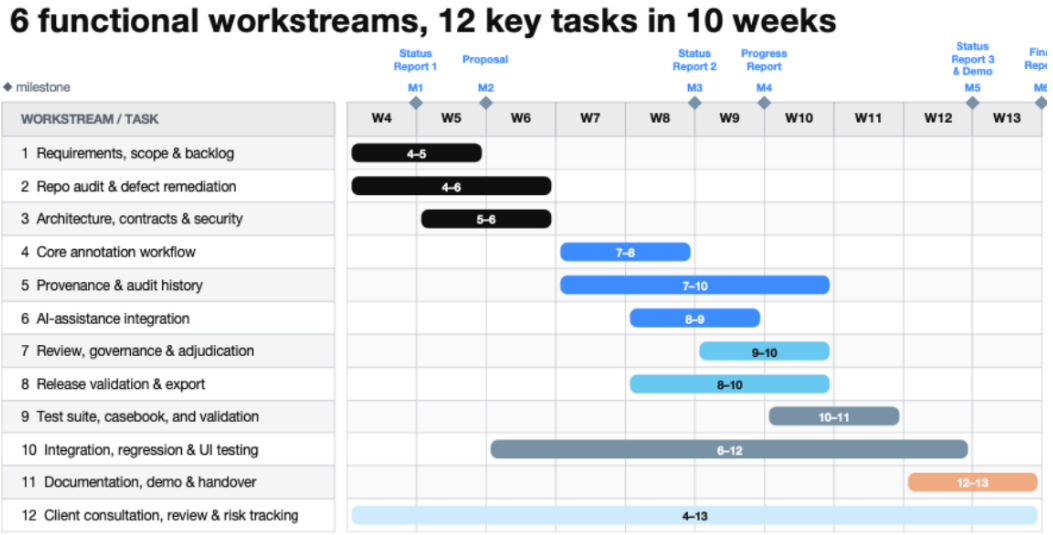


Figure 10. The original plan, reproduced from our Week 5 proposal (back to Section 3.1)

Table 14. Deviations from the proposal timeline (back to Section 3.2)

Workstream	Proposal	Actual or re-planned	Reason	Impact
1 Requirements and backlog	W4–W5	W4–W8; 59 → 63 stories	Client answers clarified scope as features were built (O3)	Four stories absorbed in W8 and the break; nothing dropped
2 Defect remediation	W4–W6, as a phase	Inside each story, W6–W11	Client direction of 26 Aug (O1)	Feature work began a week earlier; 4 defects remain, scheduled
3 Architecture and contracts	W5–W6 review	~~Separate review~~ replaced by fixing mismatches in each feature; PostgreSQL migration added in W7	A standalone review was low value; a production database was needed for concurrent work (O5)	3u of W7 went to infrastructure
4 Core annotation workflow	W7–W8	W6 – break (done)	Dependencies resolved early	Ahead; base for review work
5 Provenance and audit history	W7–W10	F3 in the break; F1 moved W7 → W8 → W11 (24 Sep) → W9 (1 Oct); F2 W9; F4–F5 W9–W10	Capacity went to queues and client-driven stories; the release gate needs F1 first (O7)	Critical path; W9 is the heaviest week (R1)
6 AI assistance	W8–W9	W6–W8 (done); mode on every item W9; blind-then-reveal W10	Built on an existing multi-provider client; blind-then-reveal redefined by the client as an evaluation protocol (21 Sep)	Ahead
7 Review, governance, adjudication	W9–W10	From W6; most done by the break; disputes paused 3–9 Oct, merging in W9; E1 → W9, E6 W10, E5 W11	Governance defects were critical (O1); dispute scope unanswered by the client (O4)	Dispute epic ends in W11, not W10 (R2)
8 Release validation and export	W8–W10	W9–W11	Depends on F1, F3, H4 and the client's release rule (answered 24 Sep)	One week later; H6 lands in the last sprint (R3)
9 Test suite and casebook	W10–W11	Harness in the break; repeatable runs W9; 40 cases and measures W10; integrity checks W11	Status Report 2 found evaluation at zero (O7)	Earlier than planned; reduces R4
10 Integration and regression testing	W6–W12	W6–W11; CI on two databases from W7; scenario testing in the break	Build ends 28 Oct	Regression testing ends with the build
11 Documentation, demo, handover	W12–W13	29 Oct – 1 Nov, then W12–W13 assessments	Week 11 made the last sprint (1 Oct)	No build buffer in W12 (R3, R7)
Sprint calendar	Mon–Sun weeks, no break work	Thu–Wed sprints; the break became a sprint	Close each sprint after the client meeting; use the break	About one extra week of capacity

A.4 Milestones and reporting

Table 15. Milestones, tasks and reporting, with changes since the proposal (back to Section 4.1)

Week	Tasks	Reporting	Status
Weeks 1–3 to 23 Aug	Project kick-off and team introductions; research on the brief; repository and workspace access (whole team)	Client meeting to review the work plan	Done
Week 4 24–30 Aug	**Milestone: Status Report 1** (26 Aug); kick-off and scope agreement; codebase review; technical audit (29 defects, 30 after a later find) (whole team)	Vision and expectation workshop	Done
Week 5 31 Aug – 6 Sep	**Milestone: Proposal** (whole team). Backlog of 59 stories (Hanchen); pull-request review workflow (Hanchen, Jingwei); ~~schema and contract audit~~ replaced: contracts fixed inside each feature	None	Done
Week 6 7–13 Sep	~~Validate contracts and schema~~; fix cross-project write scoping (G3, Jingwei); added: draft ownership (D6, Hanchen), legacy review writes retired (D2, Jingwei), AI provider integration (C1, C3, Michael)	Progress update with client	Done
Week 7 14–23 Sep	Core backend: atomic draft submission (D5, Hanchen); ~~canonical-annotation model~~ → per-author versions (F3), moved to the break; ~~task lifecycle~~ → W8; added: PostgreSQL and two-database CI (Parth), work-queue API (D8, Kanishka), project policy (B2, Jingwei), AI batch jobs (C2, Michael), invitations (G5, Tim)	Weekly client meeting; written Q&A round 1 (21 Sep); **Status Report 2 (23 Sep)**	Done; 4 tickets carried
Week 8 24–30 Sep	~~Milestone: Status Report 2~~ held on 23 Sep, the last day of sprint W7 (teaching Week 8). Dataset ingestion transactionality (B5, Kanishka, Dishank); AI module integration (C2, Michael); export/completion rule (B6, Dishank); task lifecycle (B4, Dishank); added: JSONL import (B7, Yi), annotation surfaces (B8, Yi), review names its submission (D4, Jingwei), work panel and submission chooser (D8, Hanchen), member management (J3, Tim)	Written Q&A round 2 (24 Sep)	Done; 4 tickets carried
Mid-semester break 1–7 Oct	New sprint: scenario testing (whole team; 3 bugs fixed); page load cut from 2,969 to ~560 ms (A6, Hanchen); evaluation harness (I1, Jingwei); casebook structure (I2, Hanchen, draft); answer versions (F3, Hanchen); owner reopens a final item (D9, Jingwei); blind second review (D7, Parth); available-work list (D8, Kanishka); invitee acceptance (J3, Tim); disputes (E3/E4, Yi) paused on issue #40 until 9 Oct	Client meeting (7 Oct)	Done, except E3/E4 and I2
Week 9 8–14 Oct	**Milestone: Progress Report.** Provenance events F1 (Parth), moved from W11; versioned guidelines F2 (Michael); corrections D3 and mode on every item C4 (Jingwei); one authoritative answer H4 (Dishank), from W10; release artefact H1 (Kanishka); automatic disputes E1, the dispute record E3/E4 (PR #48) and B3 check (Yi); repeatable runs I1 (Hanchen); project overview J2 and B1 check (Tim). ~~Human-review workflow; role-based governance~~ delivered in W6–break	Progress update with client (14 Oct)	Planned; 2 PRs already in review
Week 10 15–21 Oct	Release gate H3 (Dishank*) and manifest H2 (Kanishka*); dispute screens E6 (Jingwei); Expert Gate G2 (Yi*); item timeline F5 (Parth*) and actor kinds F4 (Jingwei); casebook to 40 cases I2 (Tim*) and agreement measure I3 (Parth*); AI-influence measures and blind-then-reveal protocol I4 (Michael*); project archive J4 (Tim*); reads never write A6 (Hanchen). ~~Dispute resolution workflow~~ mostly built; screens here	Client meeting to review the logic implemented	Planned
Week 11 22–28 Oct	Last build sprint. Real-data export, policy and history screens H6 (Kanishka*, first, P0); escalation posture E5 (Yi*); integrity checks I5 (Parth*) and findings record (Hanchen*); casebook toward 60 (story owners); export figure check H5 (Dishank*); J1 (Tim*, first to drop). ~~Demo-of-end-to-end integration~~ end-to-end runs continuously on AG News data; rehearsal after H6	Progress update with client	Planned
Week 12 from 29 Oct	**Milestone: Status Report 3 and demo.** No build work after 28 Oct. Technical report (Hanchen, Jingwei*), per-member decision records (every member), findings (Hanchen, Parth*) and demonstration rehearsal (Michael*), 29 Oct – 1 Nov; bug review only	Client meeting to showcase the demo	Planned
Week 13	**Milestone: Final presentation;** final report and technical report submitted	Client meeting for presentation and walkthrough	Planned

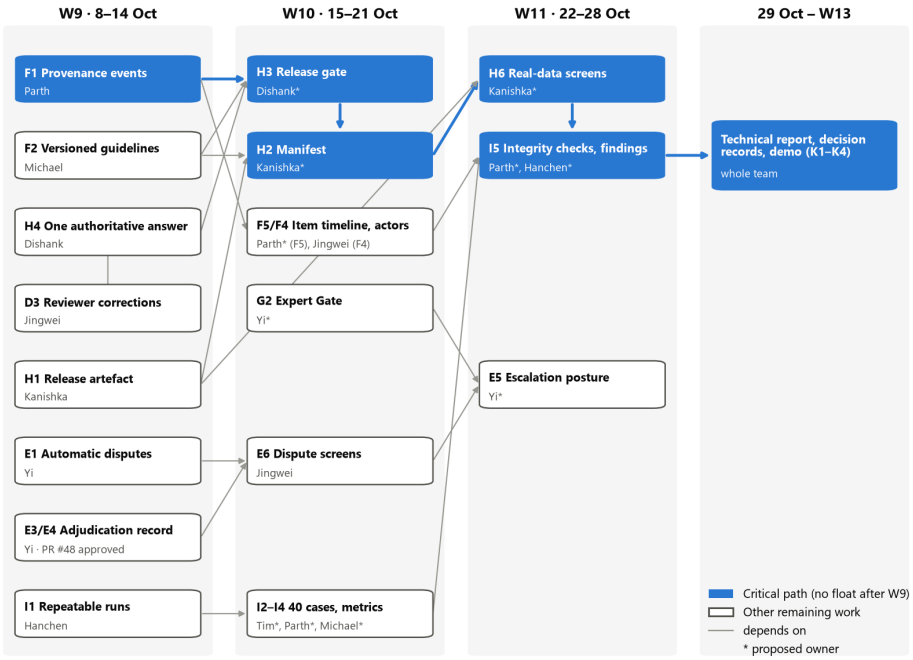


Figure 12. Dependency network of the remaining work, with owners and the critical path (back to Section 4.2)

Table 16. Remaining tickets, Weeks 9–11 (CP = critical path; * = proposed owner) (back to Section 4.2)

Week	Ticket	Work (story)	Owner	Load	Float	Depends on → unblocks
W9	SCRUM-98	Provenance event record (F1) (CP)	Parth	3u	0	F3 (done) → H3, H2, F5, I5
W9	SCRUM-53	Versioned guidelines, sources, policies (F2)	Michael	3u	0	None → H2, H3
W9	SCRUM-32	Reviewer corrections stored (D3), PR #58	Jingwei	2.5u	0	D4 (done) → H4
W9	SCRUM-37	One authoritative answer per item (H4)	Dishank	2.5u	0	F3, D4 (done); #48 reconciliation for differing answers → H3
W9	SCRUM-102	Immutable release artefact (H1)	Kanishka	2u	0	D8 list (#49, done) → H2, H6
W9	SCRUM-101	Disagreement opens a dispute (E1)	Yi	1.5u	0	D7 comparison (#53, done) → E6
W9	SCRUM-87	Annotation mode on every item (C4), PR #59	Jingwei	1u	0	None → I4
W9	SCRUM-69	Repeatable runs, regression report (I1)	Hanchen	1.5u	0	Harness (#39, done) → I3, I4
W9	SCRUM-89, 91	Project overview (J2); verify B1	Tim	2u	2 wk	B4, B6 (done)
W9	SCRUM-85	Verify B3	Yi	0.5u	2 wk	None
W9	SCRUM-41	History without repetition (F4), merged early (#57)	Jingwei	0.5u	done	None → SCRUM-40
W10	SCRUM-105	Release gate (H3) (CP)	Dishank*	2u	0	F1, F2, H4 → H2
W10	SCRUM-104	Release manifest (H2) (CP)	Kanishka*	2u	0	H3, H1, F2 → H6
W10	SCRUM-103	Dispute and arbitration screens (E6)	Jingwei	2u	0	E1, E3/E4 → E5
W10	SCRUM-73	AI-influence measures; blind-then-reveal (I4)	Michael*	3u	0	C4, I1 → I5
W10	SCRUM-71	Casebook to 40 cases (I2)	Tim*	2u	0	I1, #51 → I5
W10	SCRUM-72	Reviewer agreement measure (I3)	Parth*	1.5u	0	I1, D7 → I5
W10	SCRUM-106	An item's whole timeline on one screen (F5)	Parth*	1.5u	0	F1, F3, H4 → I5
W10	SCRUM-40	Actor kinds in the history (F4)	Jingwei	1u	0	F1 → F5, I5
W10	SCRUM-118	Expert Gate; no AI-only finalisation (G2)	Yi*	2u	0	D1 guards (done) → E5
W10	SCRUM-45	Update and archive a project (J4)	Tim* (board: Jingwei)	1.5u	1 wk	B4, H1
W10	SCRUM-121	Reads never write (A6)	Hanchen	2u	1 wk	SCRUM-99's outcome
W11	SCRUM-108	Real-data release, policy, history screens (H6), P0 (CP)	Kanishka*	2u	0	H1, H2 → I5
W11	SCRUM-74	Integrity checks on provenance, roles, releases (I5) (CP)	Parth*	1.5u	0	F4, F5, H1–H4, J3 → K3
W11	SCRUM-75	Findings record with negative results (I5) (CP)	Hanchen*	1u	0	I2–I4, SCRUM-74 → K1, K3

Week	Ticket	Work (story)	Owner	Load	Float	Depends on → unblocks
W11	SCRUM-107	Escalation posture per project (E5), P1	Yi*	2u	0	G2, E6
W11	No ticket	Export figure check (H5)	Dishank*	0.5u	0	H1–H4
W11	SCRUM-88	Stable organisation identity (J1), P2	Tim*	0.5u	drop	None

Table 17. Load per member, Weeks 9–11 and the handover days (back to Section 4.3)

Member	W9 (board)	W10	W11	29 Oct – 1 Nov
Hanchen Wang	I1 runs 1.5u	A6 reads 2u	I5 findings 1u*	Technical report lead (K1); findings (K3)*; own decision record
Jingwei Lin	D3 2.5u, C4 1u, F4 0.5u (4u)	E6 2u, F4 actors 1u (3u)	Buffer: carry-over and reviews*	Technical report co-lead*; own decision record
Yi Geng	E1 1.5u, B3 0.5u (2u)	G2 Expert Gate 2u*	E5 posture 2u*	Own decision record
Michael Max	F2 3u	I4 measures 3u*	Buffer: demonstration script on local Ollama*	Demonstration lead (K4)*; own decision record
Parth	F1 3u	F5 1.5u, I3 1.5u (3u)*	I5 integrity 1.5u*	Findings with Hanchen (K3)*; own decision record
Dishank Aswal	H4 2.5u	H3 gate 2u*	H5 check 0.5u*	Own decision record
Kanishka Kathait	H1 2u	H2 manifest 2u*	H6 screens 2u*	Own decision record
Tim Chung	J2, B1 2u	I2 cases 2u, J4 1.5u (3.5u)*	J1 0.5u*	Own decision record
Total	20u	20.5u	7.5u (+ buffer)	Whole team (K2)

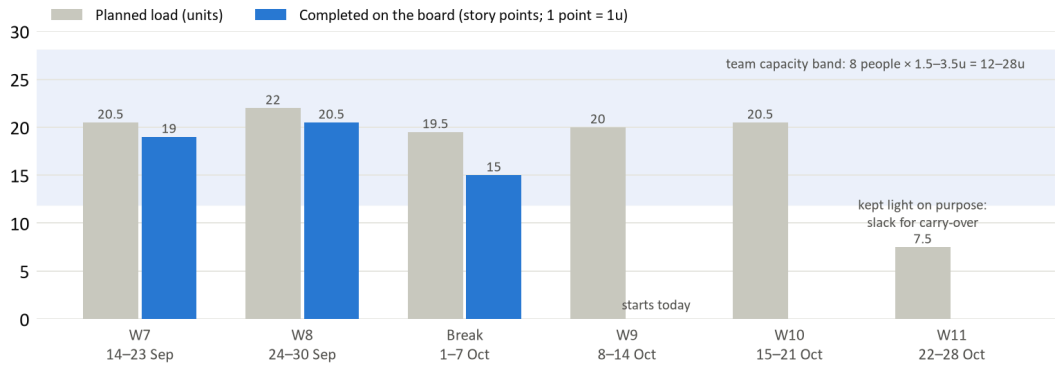


Figure 13. Planned load and completed story points by sprint, with the team's capacity band (back to Section 4.4)

Table 18. Contingencies: trigger, fallback and fall-back space (back to Section 4.4)

Trigger	Fallback	Fall-back space
F1 not merged when W9 closes (R1)	Build the gate's event-free checks first; add the provenance check when F1 merges	W10 H3 slot; W11 buffer (Jingwei, Michael)
The client's answer on #40 differs from the two-level design (R2)	Adapt in a follow-up ticket, recorded as a reversible ADR	W10 E6 slot
Any W9–W10 ticket carried over	Carry-over is planned first, then rebalanced so nobody exceeds 3.5u	W11 buffer: 4.5–20.5u of capacity
W11 still overflows	Drop J1 (0.5u), then reduce E5 (P1) by agreement with the client; never P0 or critical-path work	Released capacity goes to H6 and I5
Casebook below 40 cases when W10 closes (R4)	Each story owner adds cases for their own stories	W11 buffer
CI unavailable	Run the full suite locally and attach the result to the PR	Not needed
AI provider fails during the demo (R9)	Run on a local Ollama model or scripted answers	Demonstration rehearsal, 29 Oct – 1 Nov

Appendix B. Identifiers and Terms

Table 19. Identifiers and terms used in this report

Term	Meaning
Story, epic (e.g. D5)	A user story in the backlog, named by its epic letter (A–K) and number. Epics: A foundations, B setting up work, C AI first pass, D review and cross-validation, E disagreement and dispute, F provenance, G roles and permissions, H release and export, I evaluation, J organisations and dashboard, K evidence and handover
SCRUM-n	A ticket on the team's Jira board; a story usually has one to three tickets
#n	A pull request (or issue) in the GitHub repository USYD-CS-Capstone/hej
Unit (u), story point	Planning measure: one unit is two to three subtasks; one Jira story point equals one unit; each person carries 1.5–3.5u a week
P0, P1, P2	Story priority, from must-have (P0) to nice-to-have (P2)
Definition of Done	Demonstrable in the running app; tests in the same change; a fixed defect proven by a test that fails on the old code; documentation updated; reviewed by someone other than the author
Provenance	The recorded history of how a label came to be: source, guideline version, AI suggestion, human response, reviews and adjudication, each with an actor and a time
Authoritative answer	The one version of an item's answer that a release carries; every other version is kept as history
Cross-review	An independent, blind second review of a sampled share of submissions, used to measure agreement
Escalation, dispute, adjudication	An item reviewers cannot settle is escalated; a dispute is the record of that disagreement; adjudication is an expert's final ruling on it
Expert Gate	A project setting that holds an item after its approvals until an independent expert confirms it
Human-first, AI-assisted	Task modes: people annotate first, or the AI gives a first pass that people review
Blind-then-reveal	An evaluation protocol: the annotator answers before seeing the AI's suggestion, then may keep or revise, to measure the AI's influence
ADR	Architecture decision record: a short note of a decision, the alternatives and the reasons
Critical path	The chain of dependent work that determines the finish date; a delay on it delays the end

Appendix C. User Stories

Please use this link here to access all user stories. This GitHub Page has been maintained and updated by Hanchen Wang since Week 5. Use the export button to download a local copy.

Appendix D. Items Deprioritised or Tracked Outside Jira

User story	Item	Remarks
A1	As a project manager, I want every change to the platform to be reviewed by a second person before it goes live, because the previous team's uncoordinated changes overwrote each other and I need to trust that what I'm demonstrating is what was agreed.	This relates to project delivery practice rather than a feature of the annotation platform. It makes delivering features easier but is not essential to delivery, so it was deprioritised.
A5	As a project manager, I want a status, role or result to mean one thing across every screen, the API and the export, because the same item currently has different state names in different places and I can't tell whether two screens disagree because of a bug or because they're using different words for the same thing.	The current design already provides an indicator of the system status. A shared vocabulary is a nice-to-have feature that would help users manage items across screens, but it is not essential to the core annotation workflow and adds less value than the other core features.
K1	Technical Report	These are project deliverables rather than platform features that require technical implementation, so they are tracked outside Jira tickets.
K2	Decisions Record	
K3	Findings document	
K4	Platform Demonstration	